

PROVA G1 – TIPO 3

Gabarito

Questão Resp.

01	B
02	C
03	A
04	B
05	B
06	E
07	C
08	D
09	E
10	B
11	B
12	C
13	C
14	B
15	C
16	LSP

1 – B.

I, II e III estão corretas. POO pode ser entendida como uma abordagem dentro do grupo imperativo.

2 – C.

Impedir acesso direto ao saldo e controlar alterações por métodos é encapsulamento.

3 – A.

Transformar elementos e posteriormente filtrá-los caracteriza transformação/composição de funções.

4 – B.

Fatos + regras → inferência para responder consultas.

5 – B.

I, II e III estão corretas. Uma função pura depende essencialmente das suas entradas e não deve depender de estado externo mutável.

6 – E.

As quatro afirmações estão corretas.



7 – C.

Separar responsabilidades relacionadas em componentes especializados busca **maior coesão**.

8 – D.

Interface obriga implementadores a fornecer métodos que não fazem sentido → ISP.

9 – E.

Depender de `Repositorio`, e não de `RepositorioMySQL`, é aplicação do DIP.

10 – B.

Novos tipos de relatório devem poder ser adicionados sem modificar continuamente a classe principal → OCP.

11 – B.

I, II e III estão corretas. A IV é falsa: microservices não eliminam custos de implantação/operação.

12 – C.

Cada serviço deve controlar seus próprios dados e utilizar comunicação para acessar informações de outro serviço.

13 – C.

Nova alteração depois de V1/V2 → criar uma nova migração.

14 – B.

Perfil `dev`, `test`, `prod`, por exemplo, permite usar o mesmo código com configurações diferentes.

15 – C.

Centralização não significa atualização dinâmica automática.

Questão 16 – Tipo 3 — LSP

O enunciado deixa explícito que `Bicicleta` herda de `Veiculo`, mas não consegue cumprir adequadamente o comportamento de `acelerar()`.

a) Resposta esperada

O princípio violado é o **LSP — Liskov Substitution Principle**.

A classe `Bicicleta` herda de `Veiculo`, portanto pode ser utilizada onde um `Veiculo` é esperado:

```
Veiculo v3 = new Bicicleta();  
acelerar(v3);
```

Porém, ao chamar:



```
veiculo.acelerar();
```

a implementação de `Bicicleta` lança:

```
throw new UnsupportedOperationException(...);
```

Isso significa que `Bicicleta` **não consegue cumprir adequadamente o contrato estabelecido pela classe-base**.

Portanto, ela não pode substituir `Veiculo` de forma segura nesse contexto.

b) Solução

Uma solução é separar os veículos motorizados dos demais:

```
abstract class Veiculo {  
}  
abstract class VeiculoMotorizado extends Veiculo {  
  
    public abstract void acelerar();  
}
```

Então:

```
class Carro extends VeiculoMotorizado {  
  
    @Override  
    public void acelerar() {  
        System.out.println("Carro acelerando...");  
    }  
}  
class Moto extends VeiculoMotorizado {  
  
    @Override  
    public void acelerar() {  
        System.out.println("Moto acelerando...");  
    }  
}
```

E:

```
class Bicicleta extends Veiculo {  
}
```

Agora o método que necessita de aceleração pode receber especificamente:

```
private static void acelerar(VeiculoMotorizado veiculo) {  
    veiculo.acelerar();  
}
```

Assim:

```
acelerar(new Carro());  
acelerar(new Moto());
```

mas:

```
acelerar(new Bicicleta()); // não compila
```

E isso é desejável.

Explicação

O erro original era:

```
Veiculo
|
+-- Carro      ✓ acelera
|
+-- Moto       ✓ acelera
|
+-- Bicicleta  X não consegue cumprir acelerar()
```

A nova hierarquia fica:

```
Veiculo
|
+-- VeiculoMotorizado
|   |
|   +-- Carro
|   +-- Moto
|
+-- Bicicleta
```

Agora somente aquilo que **realmente possui o comportamento** `acelerar()` está sujeito a esse contrato.

A diferença em relação à ISP é clara:

- **ISP:** problema na **interface**, que obriga `Robo` a implementar métodos desnecessários.
- **LSP:** problema na **hierarquia de herança**, em que `Bicicleta` não consegue cumprir o contrato de `Veiculo`.